📝 “Live Scoreboard”

Component State · Event Handling · `useState`

---

1 . Problem Statement

Build a small React application that keeps track of the score in a two-team game (e.g., **Team Red vs. Team Blue**). Each team has two buttons: **“+1”** and **“-1”**. As the user clicks, the on-screen score should update instantly and never drop below 0.

---

2 . Learning Goals

Concept	Where you will practise it
`useState` hook	separate state variables for each team’s score
Event handling	click events on four buttons
Immutability / updater functions	increment & decrement without mutating directly
Derived UI	disabling the “-1” button when the score is 0
Component structure	splitting into `Scoreboard`, `TeamPanel`, and root `App`

---

3 . Requirements

ID	Requirement
R-1	**File structure** contains `App.jsx`, `Scoreboard.jsx`, `TeamPanel.jsx`, and `index.jsx`.
R-2	`App` renders a `<Scoreboard>` component and passes the team names down via props.
R-3	`Scoreboard` maintains two independent pieces of state (`redScore`, `blueScore`) using **`useState`**.
R-4	`TeamPanel` receives the current score and two callback props: `onAdd` & `onSubtract`. It renders the buttons and the score.
R-5	The **“-1” button must be disabled** whenever its team’s score is zero (illustrates conditional rendering based on state).
R-6	A **“Reset Scores”** button resets both scores to zero using a single state-update call with a functional updater.
R-7	Styling: place the two panels side-by-side with a clear visual separation; buttons should provide basic hover feedback.

ID	Requirement
R-8	Application compiles with no warnings in a standard Vite-React setup.

---

4 . Stretch Goals (Optional)

1. **Custom step** – add a numeric input that lets the user choose how many points are added or subtracted per click.
2. **History log** – keep an array of “Red +1”, “Blue -1” events and render the last five actions.
3. **Winner highlight** – when either team reaches 10 points first, show a confetti emoji 🎉 next to its name.

---

5 . Handoff Checklist

- ☐ Repository named `react-scoreboard-YOURNAME`.
- ☐ `README.md` with setup (`npm install`, `npm run dev`) and a screenshot/GIF.
- ☐ Meaningful comments explaining *why* each piece of state or handler exists.
- ☐ No ESLint or compile errors.

> **Submission format:** Zip or push the repo before the next lecture. **Tip:** Use React DevTools to watch state updates live as you test the buttons.